```
import pandas as pd
dataset = pd.read_csv("Placement.csv")
print(dataset)
```

```
     sl_no gender  ssc_p    ssc_b  hsc_p    hsc_b     hsc_s  degree_p  \
0        1      M  67.00   Others  91.00   Others  Commerce     58.00
1        2      M  79.33  Central  78.33   Others   Science     77.48
2        3      M  65.00  Central  68.00  Central      Arts     64.00
3        4      M  56.00  Central  52.00  Central   Science     52.00
4        5      M  85.80  Central  73.60  Central  Commerce     73.30
..     ...    ...    ...      ...    ...      ...       ...       ...
210    211      M  80.60   Others  82.00   Others  Commerce     77.60
211    212      M  58.00   Others  60.00   Others   Science     72.00
212    213      M  67.00   Others  67.00   Others  Commerce     73.00
213    214      F  74.00   Others  66.00   Others  Commerce     58.00
214    215      M  62.00  Central  58.00   Others   Science     53.00

      degree_t workex  etest_p specialisation  mba_p      status    salary
0     Sci&Tech     No     55.0         Mkt&HR  58.80      Placed  270000.0
1     Sci&Tech    Yes     86.5        Mkt&Fin  66.28      Placed  200000.0
2    Comm&Mgmt     No     75.0        Mkt&Fin  57.80      Placed  250000.0
3     Sci&Tech     No     66.0         Mkt&HR  59.43  Not Placed       NaN
4    Comm&Mgmt     No     96.8        Mkt&Fin  55.50      Placed  425000.0
..         ...    ...      ...            ...    ...         ...       ...
210  Comm&Mgmt     No     91.0        Mkt&Fin  74.49      Placed  400000.0
211   Sci&Tech     No     74.0        Mkt&Fin  53.62      Placed  275000.0
212  Comm&Mgmt    Yes     59.0        Mkt&Fin  69.72      Placed  295000.0
213  Comm&Mgmt     No     70.0         Mkt&HR  60.23      Placed  204000.0
214  Comm&Mgmt     No     89.0         Mkt&HR  60.22  Not Placed       NaN

[215 rows x 15 columns]
```

```
dataset.columns
```

```
Index(['sl_no', 'gender', 'ssc_p', 'ssc_b', 'hsc_p', 'hsc_b', 'hsc_s',
       'degree_p', 'degree_t', 'workex', 'etest_p', 'specialisation', 'mba_p',
       'status', 'salary'],
      dtype='object')
```

```
dataset.isnull().sum()
```

	0
sl_no	0
gender	0
ssc_p	0
ssc_b	0
hsc_p	0
hsc_b	0
hsc_s	0
degree_p	0
degree_t	0
workex	0
etest_p	0
specialisation	0
mba_p	0
status	0
salary	67

dtype: int64

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 215 entries, 0 to 214
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   sl_no           215 non-null    int64
 1   gender          215 non-null    object
 2   ssc_p           215 non-null    float64
 3   ssc_b           215 non-null    object
 4   hsc_p           215 non-null    float64
 5   hsc_b           215 non-null    object
 6   hsc_s           215 non-null    object
 7   degree_p        215 non-null    float64
 8   degree_t        215 non-null    object
 9   workex          215 non-null    object
 10  etest_p         215 non-null    float64
 11  specialisation  215 non-null    object
 12  mba_p           215 non-null    float64
 13  status          215 non-null    object
 14  salary          148 non-null    float64
dtypes: float64(6), int64(1), object(8)
memory usage: 25.3+ KB
```

```
if(dataset["ssc_b"].dtype=="object"):
  print("Qual")
else:
  print("Quan")
```

```
Qual
```

```
dataset.columns
```

```
Index(['sl_no', 'gender', 'ssc_p', 'ssc_b', 'hsc_p', 'hsc_b', 'hsc_s',
       'degree_p', 'degree_t', 'workex', 'etest_p', 'specialisation', 'mba_p',
       'status', 'salary'],
      dtype='object')
```

```
for columnName in dataset.columns:
```

```
  print(columnName)
```

```
sl_no
gender
ssc_p
ssc_b
hsc_p
hsc_b
hsc_s
degree_p
degree_t
workex
etest_p
specialisation
mba_p
status
salary
```

```
quan=[]
qual=[]
```

```
quan
```

```
[]
```

```
qual
```

```
[]
```

```
for columnName in dataset.columns:
  print(columnName)
  if(dataset[columnName].dtype=="object"):
    print("Qual")
  else:
    print("Quan")
```

```
sl_no
Quan
gender
Qual
ssc_p
Quan
ssc_b
Qual
hsc_p
Quan
hsc_b
Qual
hsc_s
Qual
degree_p
Quan
degree_t
Qual
workex
Qual
etest_p
Quan
specialisation
Qual
mba_p
Quan
status
Qual
salary
Quan
```

```
quan=[]
qual=[]
for columnName in dataset.columns:
  #print(columnName)
  if(dataset[columnName].dtype=="object"):
    #print("Qual")
    qual.append(columnName)
  else:
    #print("Quan")

    quan.append(columnName)
```

```
quan
```

```
['sl_no', 'ssc_p', 'hsc_p', 'degree_p', 'etest_p', 'mba_p', 'salary']
```

```
qual
```

```
['gender',
 'ssc_b',
 'hsc_b',
 'hsc_s',
 'degree_t',
 'workex',
 'specialisation',
 'status']
```

```
def QuanQual(dataset):
  quan=[]
  qual=[]
  for columnName in dataset.columns:
       #print(columnName)
       if(dataset[columnName].dtype=="object"):
            #print("Qual")
            qual.append(columnName)
       else:
            #print("Quan")
            quan.append(columnName)
  return quan, qual
```

```
quan,qual = QuanQual(dataset)
```

```
quan
```

```
['sl_no', 'ssc_p', 'hsc_p', 'degree_p', 'etest_p', 'mba_p', 'salary']
```

```
qual
```

```
['gender',
 'ssc_b',
 'hsc_b',
 'hsc_s',
 'degree_t',
 'workex',
 'specialisation',
 'status']
```

```
dataset["ssc_p"].mean()
```

```
np.float64(67.30339534883721)
```

```
dataset["ssc_p"].median()
```

```
67.0
```

```
dataset["ssc_p"].mode()[0]
```

```
np.float64(62.0)
```

```
dataset.describe()
```

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
count	215.000000	215.000000	215.000000	215.000000	215.000000	215.000000	148.000000
mean	108.000000	67.303395	66.333163	66.370186	72.100558	62.278186	288655.405405
std	62.209324	10.827205	10.897509	7.358743	13.275956	5.833385	93457.452420
min	1.000000	40.890000	37.000000	50.000000	50.000000	51.210000	200000.000000
25%	54.500000	60.600000	60.900000	61.000000	60.000000	57.945000	240000.000000
50%	108.000000	67.000000	65.000000	66.000000	71.000000	62.000000	265000.000000
75%	161.500000	75.700000	73.000000	72.000000	83.500000	66.255000	300000.000000
max	215.000000	89.400000	97.700000	91.000000	98.000000	77.890000	940000.000000

```
descriptive=pd.DataFrame()
```

```
print(descriptive)
```

```
Empty DataFrame
Columns: []
Index: []
```

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"])
print(descriptive)
```

```
Empty DataFrame
Columns: []
Index: [Mean, Median, Mode]
```

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"], columns=quan)
print(descriptive)
```

```
       sl_no ssc_p hsc_p degree_p etest_p mba_p salary
Mean     NaN   NaN   NaN      NaN     NaN   NaN    NaN
Median   NaN   NaN   NaN      NaN     NaN   NaN    NaN
Mode     NaN   NaN   NaN      NaN     NaN   NaN    NaN
```

```
descriptive["ssc_p"]["Median"]=0
```

```
/tmp/ipykernel_788/801119026.py:1: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive["ssc_p"]["Median"]=0
```

```
print(descriptive)
```

```
       sl_no ssc_p hsc_p degree_p etest_p mba_p salary
Mean     NaN   NaN   NaN      NaN     NaN   NaN    NaN
Median   NaN     0   NaN      NaN     NaN   NaN    NaN
Mode     NaN   NaN   NaN      NaN     NaN   NaN    NaN
```

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"], columns=quan)
for columnName in quan:
  descriptive[columnName]["Mean"] = dataset[columnName].mean()
```

```
/tmp/ipykernel_788/1505761986.py:3: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["Mean"] = dataset[columnName].mean()
```

```
print(descriptive)
```

```
        sl_no      ssc_p      hsc_p   degree_p    etest_p      mba_p  \
Mean    108.0  67.303395  66.333163  66.370186  72.100558  62.278186
Median    NaN        NaN        NaN        NaN        NaN        NaN
Mode      NaN        NaN        NaN        NaN        NaN        NaN

              salary
Mean    288655.405405
Median            NaN
Mode              NaN
```

```
dataset["ssc_p"].mean()
```

```
np.float64(67.30339534883721)
```

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"], columns=quan)
for columnName in quan:
  descriptive[columnName]["Mean"] = dataset[columnName].mean()
  descriptive[columnName]["Median"] = dataset[columnName].median()
```

```
/tmp/ipykernel_788/2010546397.py:3: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["Mean"] = dataset[columnName].mean()
/tmp/ipykernel_788/2010546397.py:4: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["Median"] = dataset[columnName].median()
```

```
print(descriptive)
```

```
        sl_no      ssc_p      hsc_p   degree_p    etest_p      mba_p  \
Mean    108.0  67.303395  66.333163  66.370186  72.100558  62.278186
Median  108.0       67.0       65.0       66.0       71.0       62.0
Mode      NaN        NaN        NaN        NaN        NaN        NaN

              salary
Mean    288655.405405
Median       265000.0
Mode              NaN
```

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"], columns=quan)
for columnName in quan:
  descriptive[columnName]["Mean"] = dataset[columnName].mean()
  descriptive[columnName]["Median"] = dataset[columnName].median()
  descriptive[columnName]["Mode"] = dataset[columnName].mode()[0]
```

```
/tmp/ipykernel_788/2513389065.py:3: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["Mean"] = dataset[columnName].mean()
/tmp/ipykernel_788/2513389065.py:4: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["Median"] = dataset[columnName].median()
/tmp/ipykernel_788/2513389065.py:5: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["Mode"] = dataset[columnName].mode()[0]
/tmp/ipykernel_788/2513389065.py:3: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["Mean"] = dataset[columnName].mean()
/tmp/ipykernel_788/2513389065.py:4: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["Median"] = dataset[columnName].median()
```

```
print(descriptive)
```

```
        sl_no      ssc_p      hsc_p   degree_p    etest_p      mba_p  \
Mean    108.0  67.303395  66.333163  66.370186  72.100558  62.278186
Median  108.0       67.0       65.0       66.0       71.0       62.0
Mode        1       62.0       63.0       65.0       60.0       56.7

               salary
Mean    288655.405405
Median       265000.0
Mode         300000.0
```

```
dataset.describe()
```

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
count	215.000000	215.000000	215.000000	215.000000	215.000000	215.000000	148.000000
mean	108.000000	67.303395	66.333163	66.370186	72.100558	62.278186	288655.405405
std	62.209324	10.827205	10.897509	7.358743	13.275956	5.833385	93457.452420
min	1.000000	40.890000	37.000000	50.000000	50.000000	51.210000	200000.000000
25%	54.500000	60.600000	60.900000	61.000000	60.000000	57.945000	240000.000000
50%	108.000000	67.000000	65.000000	66.000000	71.000000	62.000000	265000.000000
75%	161.500000	75.700000	73.000000	72.000000	83.500000	66.255000	300000.000000
max	215.000000	89.400000	97.700000	91.000000	98.000000	77.890000	940000.000000

```
print(dataset)
```

```
     sl_no gender  ssc_p    ssc_b  hsc_p    hsc_b     hsc_s  degree_p  \
0        1      M  67.00   Others  91.00   Others  Commerce     58.00
1        2      M  79.33  Central  78.33   Others   Science     77.48
2        3      M  65.00  Central  68.00  Central      Arts     64.00
3        4      M  56.00  Central  52.00  Central   Science     52.00
4        5      M  85.80  Central  73.60  Central  Commerce     73.30
..     ...    ...    ...      ...    ...      ...       ...       ...
210    211      M  80.60   Others  82.00   Others  Commerce     77.60
211    212      M  58.00   Others  60.00   Others   Science     72.00
212    213      M  67.00   Others  67.00   Others  Commerce     73.00
213    214      F  74.00   Others  66.00   Others  Commerce     58.00
214    215      M  62.00  Central  58.00   Others   Science     53.00

      degree_t workex  etest_p specialisation  mba_p      status    salary
0     Sci&Tech     No     55.0         Mkt&HR  58.80      Placed  270000.0
1     Sci&Tech    Yes     86.5        Mkt&Fin  66.28      Placed  200000.0
2    Comm&Mgmt     No     75.0        Mkt&Fin  57.80      Placed  250000.0
3     Sci&Tech     No     66.0         Mkt&HR  59.43  Not Placed       NaN
4    Comm&Mgmt     No     96.8        Mkt&Fin  55.50      Placed  425000.0
..         ...    ...      ...            ...    ...         ...       ...
210  Comm&Mgmt     No     91.0        Mkt&Fin  74.49      Placed  400000.0
211   Sci&Tech     No     74.0        Mkt&Fin  53.62      Placed  275000.0
212  Comm&Mgmt    Yes     59.0        Mkt&Fin  69.72      Placed  295000.0
213  Comm&Mgmt     No     70.0         Mkt&HR  60.23      Placed  204000.0
214  Comm&Mgmt     No     89.0         Mkt&HR  60.22  Not Placed       NaN

[215 rows x 15 columns]
```

```
import numpy as np
np.percentile(dataset["mba_p"],100)
```

```
np.float64(77.89)
```

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode","0%","25%","50%","75%","100%"], columns=quan)
for columnName in quan:
  descriptive[columnName]["Mean"] = dataset[columnName].mean()
  descriptive[columnName]["Median"] = dataset[columnName].median()
  descriptive[columnName]["Mode"] = dataset[columnName].mode()[0]
  descriptive[columnName]["0%"] = dataset.describe()[columnName]["min"]

  descriptive[columnName]["25%"] = dataset.describe()[columnName]["25%"]

  descriptive[columnName]["50%"] = dataset.describe()[columnName]["50%"]

  descriptive[columnName]["75%"] = dataset.describe()[columnName]["75%"]
  descriptive[columnName]["100%"] = dataset.describe()[columnName]["max"]
```

```
df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["50%"] = dataset.describe()[columnName]["50%"]
/tmp/ipykernel_788/1704318383.py:12: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["75%"] = dataset.describe()[columnName]["75%"]
/tmp/ipykernel_788/1704318383.py:13: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["100%"] = dataset.describe()[columnName]["max"]
```

```
print(descriptive)
```

```
        sl_no      ssc_p      hsc_p   degree_p    etest_p      mba_p  \
Mean    108.0  67.303395  66.333163  66.370186  72.100558  62.278186
Median  108.0       67.0       65.0       66.0       71.0       62.0
Mode        1       62.0       63.0       65.0       60.0       56.7
0%        1.0      40.89       37.0       50.0       50.0      51.21
25%      54.5       60.6       60.9       61.0       60.0     57.945
50%     108.0       67.0       65.0       66.0       71.0       62.0
75%     161.5       75.7       73.0       72.0       83.5     66.255
100%    215.0       89.4       97.7       91.0       98.0      77.89

               salary
Mean    288655.405405
Median       265000.0
Mode         300000.0
0%           200000.0
25%          240000.0
50%          265000.0
75%          300000.0
100%         940000.0
```

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode","0%","25%","50%","75%","100%","IQR","1.6*IQR","LesserOutlier","GreaterOutlier"], columns=quan
for columnName in quan:
  descriptive[columnName]["Mean"] = dataset[columnName].mean()
  descriptive[columnName]["Median"] = dataset[columnName].median()
  descriptive[columnName]["Mode"] = dataset[columnName].mode()[0]
  descriptive[columnName]["0%"] = dataset.describe()[columnName]["min"]

  descriptive[columnName]["25%"] = dataset.describe()[columnName]["25%"]

  descriptive[columnName]["50%"] = dataset.describe()[columnName]["50%"]

  descriptive[columnName]["75%"] = dataset.describe()[columnName]["75%"]
  descriptive[columnName]["100%"] = dataset.describe()[columnName]["max"]
  descriptive[columnName]["IQR"] = descriptive[columnName]["75%"] - descriptive[columnName]["25%"]

  descriptive[columnName]["1.6*IQR"] = 1.6 * descriptive[columnName]["IQR"]
  descriptive[columnName]["LesserOutlier"] = descriptive[columnName]["25%"] - descriptive[columnName]["1.6*IQR"]


  descriptive[columnName]["GreaterOutlier"] = descriptive[columnName]["75%"] + descriptive[columnName]["1.6*IQR"]
```

```
df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["LesserOutlier"] = descriptive[columnName]["25%"] - descriptive[columnName]["1.6*IQR"]
/tmp/ipykernel_788/743838617.py:20: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["GreaterOutlier"] = descriptive[columnName]["75%"] + descriptive[columnName]["1.6*IQR"]
```

```
print(descriptive)
```

```
                sl_no      ssc_p      hsc_p   degree_p    etest_p      mba_p  \
Mean           108.0  67.303395  66.333163  66.370186  72.100558  62.278186
Median         108.0       67.0       65.0       66.0       71.0       62.0
Mode               1       62.0       63.0       65.0       60.0       56.7
0%               1.0      40.89       37.0       50.0       50.0      51.21
25%             54.5       60.6       60.9       61.0       60.0     57.945
50%            108.0       67.0       65.0       66.0       71.0       62.0
75%            161.5       75.7       73.0       72.0       83.5     66.255
100%           215.0       89.4       97.7       91.0       98.0      77.89
IQR            107.0       15.1       12.1       11.0       23.5       8.31
1.6*IQR        171.2      24.16      19.36       17.6       37.6     13.296
LesserOutlier -116.7      36.44      41.54       43.4       22.4     44.649
GreaterOutlier  332.7      99.86      92.36       89.6      121.1     79.551

                      salary
Mean            288655.405405
Median               265000.0
Mode                 300000.0
0%                   200000.0
25%                  240000.0
50%                  265000.0
75%                  300000.0
100%                 940000.0
IQR                   60000.0
1.6*IQR               96000.0
LesserOutlier        144000.0
GreaterOutlier       396000.0
```

```
if(descriptive["hsc_p"]["LesserOutlier"] > descriptive["hsc_p"]["0%"]):
  print("Outlier")
else:
  print("No outlier")
```

```
Outlier
```

```
Lesser = []
Greater=[]
for columnNmae in quan:
  if(descriptive[columnName] ["LesserOutlier"] > descriptive[columnName]["0%"]):
    Lesser.append(columnName)
  if(descriptive[columnName] ["GreaterOutlier"] < descriptive[columnName]["100%"]):
    Greater.append(columnName)
```

```
Lesser
```

```
[]
```

```
Greater
```

```
['salary', 'salary', 'salary', 'salary', 'salary', 'salary', 'salary']
```

```
for LesserColumn in Lesser:
  dataset[LesserColumn][dataset[LesserColumn]<descriptive[LesserColumn]["LesserOutlier"]] = descriptive[LesserColumn]["LesserOutlier"]
for GreaterColumn in Greater:
    dataset[GreaterColumn][dataset[GreaterColumn]>descriptive[GreaterColumn]["GreaterOutlier"]] = descriptive[GreaterColumn]["GreaterOutlier"]
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  dataset[GreaterColumn][dataset[GreaterColumn]>descriptive[GreaterColumn]["GreaterOutlier"]] = descriptive[GreaterColumn]["GreaterOutlier"]
/tmp/ipykernel_788/2229719424.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  dataset[GreaterColumn][dataset[GreaterColumn]>descriptive[GreaterColumn]["GreaterOutlier"]] = descriptive[GreaterColumn]["GreaterOutlier"]
/tmp/ipykernel_788/2229719424.py:4: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  dataset[GreaterColumn][dataset[GreaterColumn]>descriptive[GreaterColumn]["GreaterOutlier"]] = descriptive[GreaterColumn]["GreaterOutlier"]
/tmp/ipykernel_788/2229719424.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode","0%","25%","50%","75%","100%","IQR","1.6*IQR","LesserOutlier","GreaterOutlier"], columns=quan)
for columnName in quan:
  descriptive[columnName]["Mean"] = dataset[columnName].mean()
  descriptive[columnName]["Median"] = dataset[columnName].median()
  descriptive[columnName]["Mode"] = dataset[columnName].mode()[0]
  descriptive[columnName]["0%"] = dataset.describe()[columnName]["min"]

  descriptive[columnName]["25%"] = dataset.describe()[columnName]["25%"]

  descriptive[columnName]["50%"] = dataset.describe()[columnName]["50%"]

  descriptive[columnName]["75%"] = dataset.describe()[columnName]["75%"]
  descriptive[columnName]["100%"] = dataset.describe()[columnName]["max"]
  descriptive[columnName]["IQR"] = descriptive[columnName]["75%"] - descriptive[columnName]["25%"]

  descriptive[columnName]["1.6*IQR"] = 1.6 * descriptive[columnName]["IQR"]
  descriptive[columnName]["LesserOutlier"] = descriptive[columnName]["25%"] - descriptive[columnName]["1.6*IQR"]


  descriptive[columnName]["GreaterOutlier"] = descriptive[columnName]["75%"] + descriptive[columnName]["1.6*IQR"]
```

```
  descriptive[columnName]["75%"] = dataset.describe()[columnName]["75%"]
/tmp/ipykernel_788/743838617.py:13: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["100%"] = dataset.describe()[columnName]["max"]
/tmp/ipykernel_788/743838617.py:14: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["IQR"] = descriptive[columnName]["75%"] - descriptive[columnName]["25%"]
/tmp/ipykernel_788/743838617.py:16: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["1.6*IQR"] = 1.6 * descriptive[columnName]["IQR"]
/tmp/ipykernel_788/743838617.py:17: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["LesserOutlier"] = descriptive[columnName]["25%"] - descriptive[columnName]["1.6*IQR"]
/tmp/ipykernel_788/743838617.py:20: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  descriptive[columnName]["GreaterOutlier"] = descriptive[columnName]["75%"] + descriptive[columnName]["1.6*IQR"]
```

```
print(descriptive)
```

```
               sl_no      ssc_p      hsc_p   degree_p    etest_p      mba_p  \
Mean           108.0  67.303395  66.333163  66.370186  72.100558  62.278186
Median         108.0       67.0       65.0       66.0       71.0       62.0
Mode               1       62.0       63.0       65.0       60.0       56.7
0%               1.0      40.89       37.0       50.0       50.0      51.21
```

```
25%              54.5       60.6       60.9       61.0       60.0     57.945
50%             108.0       67.0       65.0       66.0       71.0       62.0
75%             161.5       75.7       73.0       72.0       83.5     66.255
100%            215.0       89.4       97.7       91.0       98.0      77.89
IQR             107.0       15.1       12.1       11.0       23.5       8.31
1.6*IQR         171.2      24.16      19.36       17.6       37.6     13.296
LesserOutlier  -116.7      36.44      41.54       43.4       22.4     44.649
GreaterOutlier  332.7      99.86      92.36       89.6      121.1     79.551

                       salary
Mean             278236.486486
Median                265000.0
Mode                  300000.0
0%                    200000.0
25%                   240000.0
50%                   265000.0
75%                   300000.0
100%                  396000.0
IQR                    60000.0
1.6*IQR                96000.0
LesserOutlier         144000.0
GreaterOutlier        396000.0
```

```
Lesser1 = []
Greater1=[]
for columnNmae in quan:
  if(descriptive[columnName] ["LesserOutlier"] > descriptive[columnName]["0%"]):
    Lesser.append(columnName)
  if(descriptive[columnName] ["GreaterOutlier"] < descriptive[columnName]["100%"]):
    Greater.append(columnName)
```

```
Lesser1
```

```
[]
```

```
Greater1
```

```
[]
```

```
print(descriptive)
```

```
                sl_no      ssc_p      hsc_p   degree_p    etest_p      mba_p  \
Mean            108.0  67.303395  66.333163  66.370186  72.100558  62.278186
Median          108.0       67.0       65.0       66.0       71.0       62.0
Mode                1       62.0       63.0       65.0       60.0       56.7
0%                1.0      40.89       37.0       50.0       50.0      51.21
25%              54.5       60.6       60.9       61.0       60.0     57.945
50%             108.0       67.0       65.0       66.0       71.0       62.0
75%             161.5       75.7       73.0       72.0       83.5     66.255
100%            215.0       89.4       97.7       91.0       98.0      77.89
IQR             107.0       15.1       12.1       11.0       23.5       8.31
1.6*IQR         171.2      24.16      19.36       17.6       37.6     13.296
LesserOutlier  -116.7      36.44      41.54       43.4       22.4     44.649
GreaterOutlier  332.7      99.86      92.36       89.6      121.1     79.551

                       salary
Mean             278236.486486
Median                265000.0
Mode                  300000.0
0%                    200000.0
25%                   240000.0
50%                   265000.0
75%                   300000.0
100%                  396000.0
IQR                    60000.0
1.6*IQR                96000.0
LesserOutlier         144000.0
GreaterOutlier        396000.0
```

```
dataset.shape
print("Total rows:",dataset.shape[0])
print("Total columns:",dataset.shape[1])
print("Total rows and columns:",dataset.shape)
```

```
Total rows: 215
Total columns: 15
Total rows and columns: (215, 15)
```

```
dataset["ssc_p"].value_counts()
```

ssc_p	count
62.00	11
63.00	10
67.00	9
73.00	9
52.00	9
...	...
55.68	1
41.00	1
83.33	1
43.00	1
80.60	1

103 rows × 1 columns

dtype: int64

```
#univarient

import seaborn as sns
```

```
import matplotlib.pyplot as plt
sns.distplot(dataset['ssc_p'],hist=True,kde=True,kde_kws={'color':'red'})
```

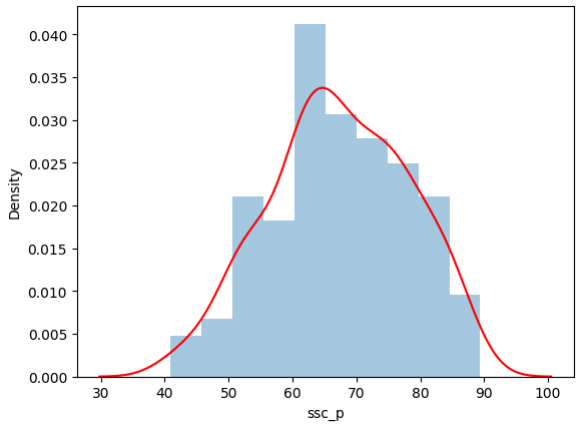
```
/tmp/ipykernel_788/1554776786.py:5: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

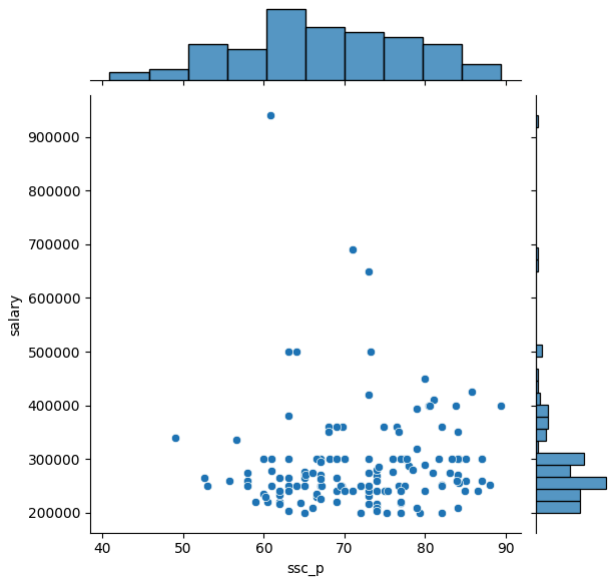
Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

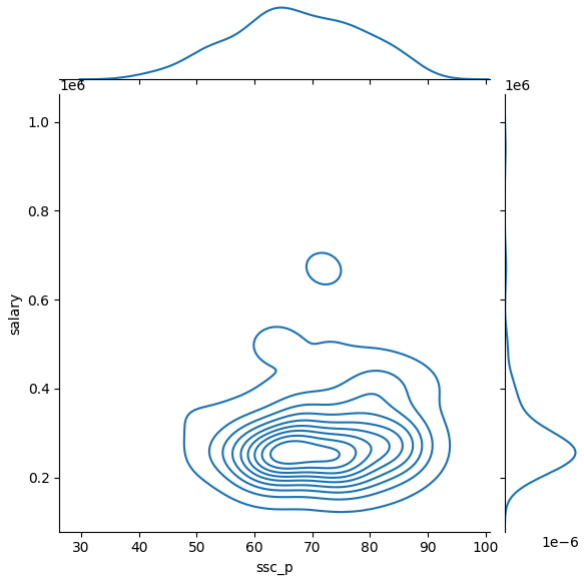
  sns.distplot(dataset['ssc_p'],hist=True,kde=True,kde_kws={'color':'red'})
<Axes: xlabel='ssc_p', ylabel='Density'>
```




```
#Bivarient
sns.jointplot(x = 'ssc_p',y = 'salary',data = dataset)
plt.show()
```




```
sns.jointplot(x = 'ssc_p',y = 'salary',data = dataset,kind='kde')
plt.show()
```

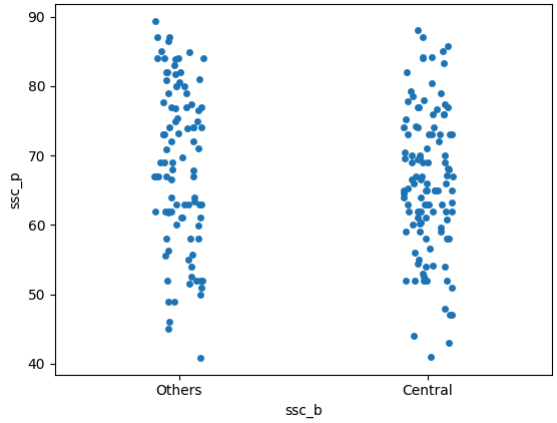


```
#pair plot
sns.pairplot(dataset,hue = 'gender',diag_kind = "kde",kind = "scatter",palette = "husl")
plt.show()
```

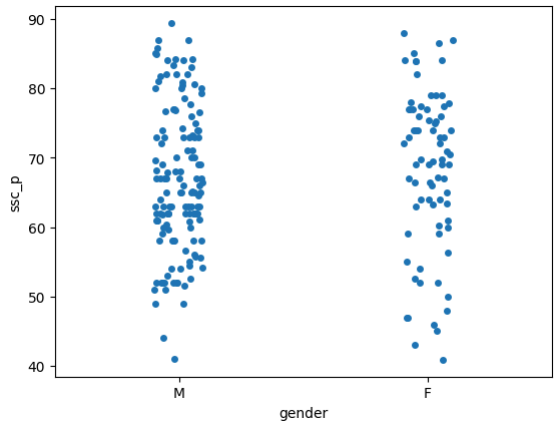



```
#categorical plt

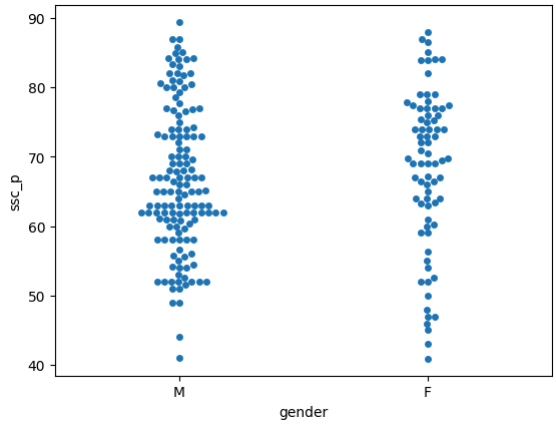
sns.stripplot(x = "ssc_b", y = "ssc_p", data = dataset)
plt.show()
```




```
sns.stripplot(x = "gender", y = "ssc_p", data = dataset,jitter=True)
plt.show()
```

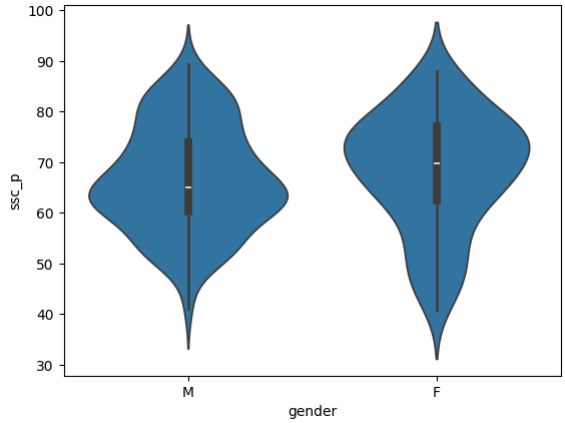



```
sns.swarmplot(x = "gender", y = "ssc_p", data = dataset)
plt.show()
```

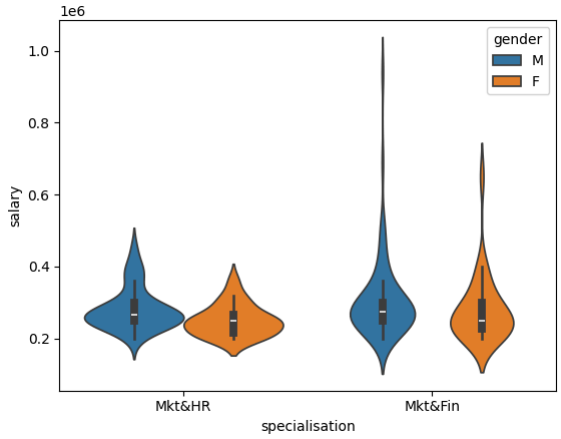



```
#Distribution observation

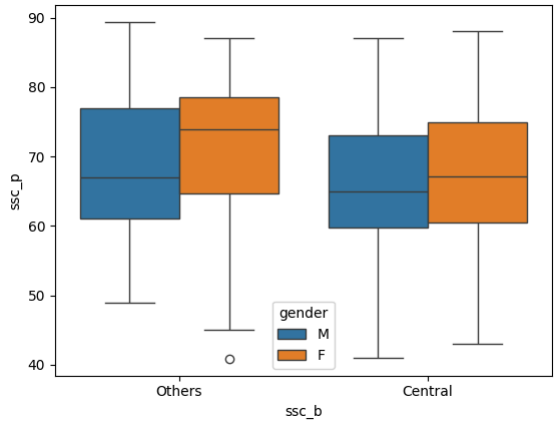
sns.violinplot(x = "gender", y = "ssc_p", data = dataset)
plt.show()
```



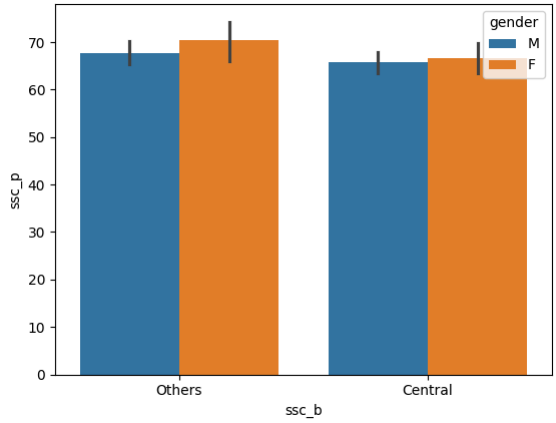

```
sns.violinplot(x = "specialisation", y = "salary", data = dataset,hue="gender")
plt.show()
```



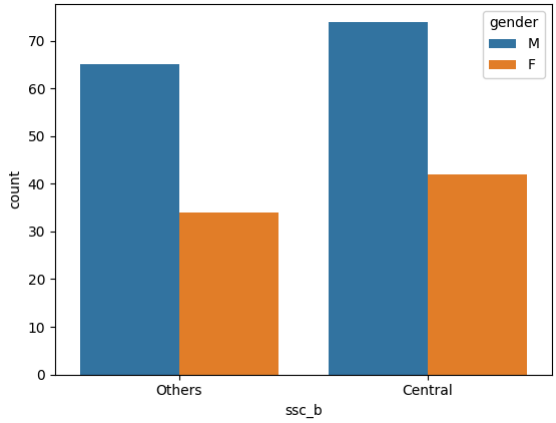
```
sns.boxplot(x = "ssc_b", y = "ssc_p", data = dataset,hue="gender" )
plt.show()
```



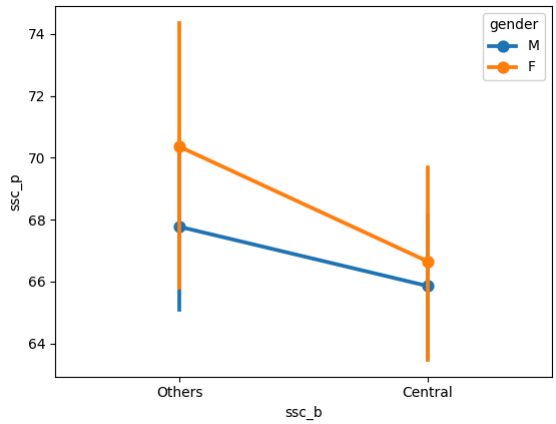

```
#Bar plot
sns.barplot(x = "ssc_b", y = "ssc_p", data = dataset,hue="gender" )
plt.show()
```




```
sns.countplot(x ="ssc_b",data = dataset,hue="gender")
plt.show()
```




```
sns.pointplot(x = "ssc_b", y = "ssc_p", data = dataset,hue="gender" )
plt.show()
```



```
sns.boxplot(data = dataset,orient="h")
plt.show()
```

